

Đặc tả: Xử lý Thanh toán (Payment Processing - Circuit Breaker & Idempotency)

1. Mô tả

Hệ thống thanh toán đối với các workshop có thu phí sẽ tích hợp với một **Cổng thanh toán giả lập (Mock Payment Gateway)**. Trọng tâm của tính năng này không phải là việc trừ tiền thật, mà là việc thiết lập 2 cơ chế bảo vệ backend cốt lõi:

Idempotency Key: Chống trừ tiền hai lần (Double Charge) khi client gặp lỗi mạng và gửi lại yêu cầu (Retry).

Circuit Breaker (Cầu dao tự động): Ngăn chặn hệ thống liên tục gọi đến Payment Gateway khi gateway này bị sập (down), giúp fail-fast và tự động hạ cấp dịch vụ (Graceful Degradation) để không làm treo toàn bộ server UniHub.

2. Luồng chính (Happy Path)

Luồng 2.1. Thanh toán hoàn chỉnh (Standard Flow)

Bối cảnh: Sinh viên đã đăng ký giữ chỗ workshop có phí, bản ghi Registration đang ở trạng thái PENDING.

B1. Khởi tạo định danh: Tại trang chi tiết vé, khi sinh viên bấm [Thanh toán], Frontend tự động tạo một Idempotency Key (chuỗi UUID v4) và gắn vào Header X-Idempotency-Key của request.

B2. Kiểm tra Idempotency: Backend ưu tiên kiểm tra Key này trong Cache (Redis). Nếu chưa tồn tại, kiểm tra tiếp trong Database để đảm bảo giao dịch này chưa từng được xử lý.

B3. Khởi tạo giao dịch: Backend tạo bản ghi trong bảng Payments với trạng thái PENDING.

B4. Gọi Gateway (Qua Circuit Breaker): Hệ thống gọi đến API của Mock Payment Gateway thông qua một lớp vỏ bọc là Circuit Breaker (đang ở trạng thái CLOSED - Cho phép đi qua).

B5. Cập nhật trạng thái: Gateway trả về thành công. Backend cập nhật trạng thái Payments thành SUCCESS và Registrations thành PAID.

B6. Lưu vết & Trả kết quả: Lưu Idempotency Key vào Redis với TTL 24h. Trả về HTTP 200 OK cho Frontend để hiển thị mã QR. Đẩy tác vụ gửi Email biên lai vào Background Queue.

Luồng 2.2. Kích hoạt Graceful Degradation (Hạ cấp dịch vụ khi Gateway sập)

Bối cảnh: Mock Payment Gateway đang bị lỗi diện rộng, Circuit Breaker đã chuyển sang trạng thái OPEN (Ngắt mạch).

B1. Chặn request: Request thanh toán của sinh viên bị Circuit Breaker chặn lại ngay lập tức ở tầng Backend (không tốn thời gian chờ Gateway phản hồi).

B2. Xử lý hạ cấp (Graceful Degradation): Thay vì văng lỗi bắt sinh viên thử lại, Backend giữ nguyên trạng thái PENDING, đồng thời đẩy một tác vụ RetryPaymentJob vào Redis Queue (BullMQ) với cơ chế Exponential Backoff (thử lại ngằm sau 1p, 2p, 4p...).

B3. Phản hồi mượt mà: Trả về HTTP Status 202 Accepted. Frontend hiển thị thông báo: “Hệ thống thanh toán đang quá tải. Giao dịch của bạn đang được xử lý, mã vé sẽ được gửi qua email ngay khi hoàn tất.”

3. Kịch bản lỗi (Edge Cases & Exception Handling)

3.1. Client retry do lag mạng (CÙNG Idempotency Key)

- **Trigger:** Sinh viên bấm thanh toán, mạng lag không thấy phản hồi nên bấm liên tục 3-4 lần. Các request này mang cùng một mã X-Idempotency-Key.
- **Xử lý:** Backend phát hiện Key ĐÃ TỒN TẠI trong Redis Cache. Lập tức chặn các request đến sau và trả về nguyên xi kết quả đã cache của request đầu tiên (HTTP 200 OK). Gateway không bị gọi 2 lần, tuyệt đối không có double charge.

3.2. Client cố tình làm mới trang (KHÁC Idempotency Key)

- **Trigger:** Sinh viên F5 trang web và bấm nút thanh toán lại, Frontend sinh ra một Idempotency Key mới tinh.
- **Xử lý:** Backend đi qua cửa ải Redis nhưng bị chặn lại ở tầng Database: Truy vấn bảng Registrations phát hiện vé này đã được thanh toán (payment_status = PAID). Trả về HTTP 409 Conflict - “Vé của bạn đã được thanh toán thành công trước đó.”

3.3. Chuyển đổi trạng thái Circuit Breaker

- **Trigger:** Mock Gateway bị lỗi timeout liên tục.
- **Xử lý:** Nếu đếm được 5 lỗi trong 60 giây -> Đổi trạng thái từ CLOSED sang OPEN (Bắt đầu chặn request).
 - + Sau khi OPEN được 30 giây -> Đổi sang HALF_OPEN. Cho phép ĐÚNG 1 request đi lọt qua để thăm dò.
 - + Nếu request thăm dò thành công -> Gateway đã hồi sinh, chuyển lại về CLOSED. Nếu thất bại -> Tiếp tục OPEN.

4. Ràng buộc (Constraints)

- **Quy chuẩn Idempotency Key:** Bắt buộc sử dụng định dạng UUID v4. Key được lưu trữ trên Redis với thời gian sống (TTL) là 24 giờ.
- **Ngưỡng chịu đựng (Circuit Breaker Threshold):** Failure threshold: 5 lỗi / 60 giây. Reset timeout: 30 giây.
- **Tính Bất biến (Immutability):** Dữ liệu trong bảng Payments là bất biến một chiều. Tuyệt đối không cho phép cập nhật trạng thái từ SUCCESS ngược trở lại thành PENDING hay FAILED.
- **Mô phỏng (Mocking):** Xây dựng sẵn API ẩn (Admin only) để cố tình điều khiển Mock Gateway: POST /admin/payment/set-failure-rate nhằm phục vụ việc demo và chấm điểm.

5. Tiêu chí chấp nhận (Acceptance Criteria)

- **Test Case 1 (Happy Path):** Gọi API thanh toán với Key A hợp lệ. Giao dịch thành công, sinh ra mã QR.
- **Test Case 2 (Idempotency):** Gọi API thanh toán 5 lần liên tiếp trong 1 giây với CÙNG một Key A. Hệ thống chỉ xử lý 1 lần, không bị văng lỗi, không sinh ra 5 giao dịch trùng lặp.

- **Test Case 3 (Resilience):** Dùng API ẩn set tỷ lệ lỗi của Mock Gateway lên 100%. Gọi API thanh toán 5 lần (nhận lỗi). Ở lần thứ 6, hệ thống phải ngắt mạch (Circuit OPEN) và thời gian phản hồi ở lần 6 phải cực nhanh ($< 10\text{ms}$) do bị chặn ngay tại Backend.
- **Test Case 4 (Recovery):** Chờ 30 giây sau Test Case 3, hệ thống chuyển sang Half-Open. Đặt tỷ lệ lỗi Gateway về 0%. Gọi API thanh toán, giao dịch thành công và Circuit Breaker khôi phục về Closed.